

DBMS – ORDER AND ORDER_ITEM MANAGEMENT

TITAN EYE – EYEWEAR E-COMMERCE DATABASE SYSTEM

1. DATABASE DESIGN

1.1 ORDERS TABLE

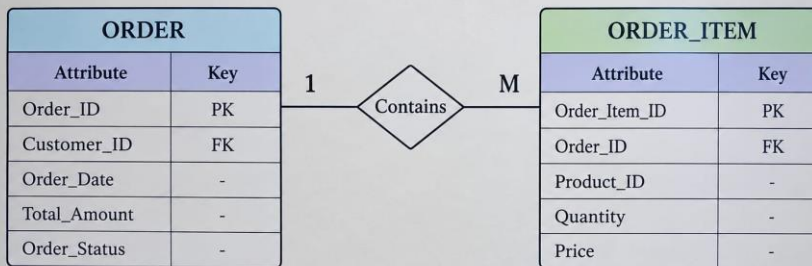
Attribute	Description
Order_ID	Unique order ID
Customer_ID	Customer ID
Order_Date	Date of order
Total_Amount	Total order amount
Order_Status	Current order status

1.2 ORDER_ITEM TABLE

Attribute	Description
Order_Item_ID	Unique order item ID
Order_ID	Order ID (FK)
Product_ID	Product ID
Quantity	Quantity ordered
Price	Product price

2 .RELATIONSHIP

2. RELATIONSHIP



Relationship Explanation:

- ORDER → ORDER_ITEM
- Cardinality : One-to-Many (1:M)
- One order can contain multiple order items, while each order item belongs to one order.
- Order_ID in the ORDER_ITEM table acts as a Foreign Key referencing Order_ID in the ORDER table.

3.TABLE CREATION

3.1 ORDER TABLE

```
SQL> CREATE TABLE Orders (  
2     Order_ID INT PRIMARY KEY,  
3     Customer_ID INT,  
4     Order_Date DATE,  
5     Total_Amount DECIMAL(10,2),  
6     Order_Status VARCHAR(30)  
7 );
```

Table created.

3.2 ORDER_ITEM TABLE

```
SQL> CREATE TABLE Order_Item (  
2     Order_Item_ID INT PRIMARY KEY,  
3     Order_ID INT,  
4     Product_ID INT,  
5     Quantity INT,  
6     Price DECIMAL(10,2),  
7     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
8 );
```

Table created.

4.ORDER TABLE DATA INSERTION

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2 VALUES (101, 1, DATE '2026-09-01', 4500.00, 'Delivered');

1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2 VALUES (102, 2, DATE '2026-09-03', 2999.00, 'Shipped');

1 row created.

SQL>
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2 VALUES (103, 3, DATE '2026-09-05', 5999.00, 'Processing');

1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2 VALUES (104, 4, DATE '2026-09-07', 1999.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2 VALUES (105, 5, DATE '2026-09-10', 7499.00, 'Pending');

1 row created.
```

4.1 DISPLAY ORDER TABLE DATA

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
101	1	01-SEP-26	4500	Delivered
102	2	03-SEP-26	2999	Shipped
103	3	05-SEP-26	5999	Processing
104	4	07-SEP-26	1999	Delivered
105	5	10-SEP-26	7499	Pending

5.ORDER_ITEM TABLE DATA INSERTION

SELECT * FROM Customer;

```
SQL> INSERT INTO Order_Item (Order_Item_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES (1, 101, 201, 1, 4500.00);

1 row created.

SQL> INSERT INTO Order_Item (Order_Item_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES (2, 102, 202, 1, 2999.00);

1 row created.

SQL> INSERT INTO Order_Item (Order_Item_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES (3, 103, 203, 1, 5999.00);

1 row created.

SQL> INSERT INTO Order_Item (Order_Item_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES (4, 104, 204, 1, 1999.00);

1 row created.

SQL> INSERT INTO Order_Item (Order_Item_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES (5, 105, 205, 1, 7499.00);

1 row created.
```

5.1 DISPLAY ORDER_ITEM TABLE DATA

```
SQL> SELECT * FROM Order_Item;
```

ORDER_ITEM_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
1	101	201	1	4500
2	102	202	1	2999
3	103	203	1	5999
4	104	204	1	1999
5	105	205	1	7499

6.UPDATE ORDERS TABLE

6.1 UPDATE ORDER STATUS:

```
SQL> UPDATE Orders
  2 SET Order_Status = 'Shipped'
  3 WHERE Order_ID = 101;

1 row updated.
```

6.2 UPDATE TOTAL AMOUNT:

```
SQL> UPDATE Orders
  2  SET Total_Amount = 4999.00
  3  WHERE Order_ID = 102;

1 row updated.
```

7. UPDATE ORDER_ITEM TABLE

7.1 UPDATE QUANTITY:

```
SQL> UPDATE Order_Item
  2  SET Quantity = 2
  3  WHERE Order_Item_ID = 1;

1 row updated.
```

7.2 UPDATE PRICE:

```
SQL> UPDATE Order_Item
  2  SET Price = 4800.00
  3  WHERE Order_Item_ID = 2;

1 row updated.
```

8.DISPLAY THE UPDATED TABLES

```
SQL> SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
101	1	01-SEP-26	4500	Shipped
102	2	03-SEP-26	4999	Shipped
103	3	05-SEP-26	5999	Processing
104	4	07-SEP-26	1999	Delivered
105	5	10-SEP-26	7499	Pending

```
SQL> SELECT * FROM Order_Item;
```

ORDER_ITEM_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
1	101	201	2	4500
2	102	202	1	4800
3	103	203	1	5999
4	104	204	1	1999
5	105	205	1	7499

```
SQL> COMMIT;
```

```
Commit complete.
```

9 CUSTOMER ORDER HISTORY REPORT

```
SQL> SELECT
2     o.Customer_ID,
3     o.Order_ID,
4     o.Order_Date,
5     oi.Product_ID,
6     oi.Quantity,
7     oi.Price,
8     o.Total_Amount
9 FROM Orders o
10 JOIN Order_Item oi
11     ON o.Order_ID = oi.Order_ID
12 ORDER BY o.Customer_ID, o.Order_Date;
```

CUSTOMER_ID	ORDER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	PRICE	TOTAL_AMOUNT
1	101	01-SEP-26	201	2	4500	4500
2	102	03-SEP-26	202	1	4800	4999
3	103	05-SEP-26	203	1	5999	5999
4	104	07-SEP-26	204	1	1999	1999
5	105	10-SEP-26	205	1	7499	7499

```
SQL>
```

10.CUSTOMER-WISE ORDER SUMMARY

```
SQL> SELECT
  2     o.Customer_ID AS Customer_ID,
  3     COUNT(o.Order_ID) AS Total_Orders,
  4     SUM(o.Total_Amount) AS Total_Amount
  5 FROM Orders o
  6 GROUP BY o.Customer_ID
  7 ORDER BY o.Customer_ID;
```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_AMOUNT
1	1	4500
2	1	4999
3	1	5999
4	1	1999
5	1	7499